INDEX

SR. NO	AIM	Date of Experiment	Date of Submission
1	Breadth First Search & Iterative Depth First Search .	19/06/2025	05/07/2025
2	A* Search and Recursive Best-First Search	28/06/2025	24/07/2025
3	Decision Tree Learning	17/07/2025	24/07/2025
4	Feed Forward Backpropagation Neural Network	24/07/2025	07/08/2025
5	Support Vector Machines (SVM)	31/07/2025	07/08/2025
6	Adaboost Ensemble Learning	07/08/2025	21/08/2025
7	Naive Bayes' Classifier	14/08/2025	21/08/2025
8	K-Nearest Neighbors (K-NN)	21/08/2025	25/09/2025
9	Association Rule Mining	11/09/2025	25/09/2025
10	Demo of OpenAI/TensorFlow Tools	18/09/2025	25/09/2025

PRACTICAL NO 1

Breadth First Search & Iterative Depth First Search

Implement the Breadth First Search algorithm to solve a given problem

Breadth-First Search (BFS) is a fundamental algorithm used to traverse a tree in a **level-wise manner**. In this method, nodes are visited one level at a time, starting from the **root node**, then moving on to all nodes at the next level, and so on until every node in the tree has been visited.

The traversal begins at the root node, which is the topmost node in the tree. From the root, the algorithm first visits all immediate child nodes. Once all nodes at the current level are processed, the traversal continues to the nodes at the next level. This process continues level by level until all nodes have been visited.

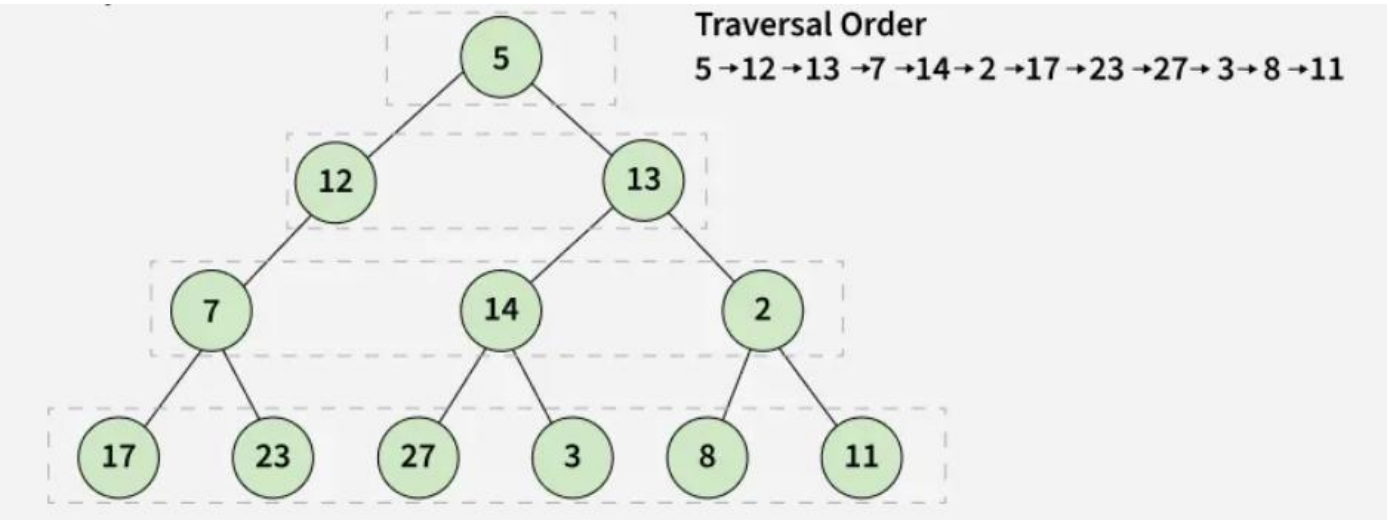
BFS is implemented using a **queue** data structure, which follows the **First-In-First-Out (FIFO)** principle. At the beginning of the traversal, the root node is inserted into the queue. Then, in a loop, the node at the front of the queue is removed (dequeued), processed, and its children are added (enqueued) to the end of the queue. This ensures that nodes are processed in the exact order they appear level by level in the tree.

One of the key benefits of BFS in trees is that it **does not require tracking visited nodes**, because in a tree, each node is unique and there are no repeated connections or cycles. This makes BFS implementation in trees simpler and more efficient compared to other structures.

The **time complexity** of BFS in a tree is **O(n)**, where n is the number of nodes in the tree. This is because each node is visited exactly once. The **space complexity** is also **O(n)** in the worst case, especially when the tree is wide and a large number of nodes need to be stored in the queue at the same time (for example, all nodes at the bottom level).

BFS is particularly useful when solving problems that involve:

- Visiting nodes **in level order**
- Finding the **minimum number of steps** from the root to a particular node
- **Determining the height** or **depth** of a tree
- Collecting or printing nodes grouped by level



Applications of BFS

File System Browsers (Windows Explorer, macOS Finder)

Operating systems organize files and directories in a **tree-like structure**, where each folder is a node, and subfolders or files are its children. When displaying the contents of a folder, BFS can be used to **traverse level by level**, such as

showing all folders at a given level before going deeper into subfolders. This approach helps efficiently organize and display file hierarchies.

Auto-suggestion and Predictive Text (Search Engines, Keyboards)

Auto-complete features (like Google Search or smartphone keyboards) often use **prefix trees (tries)** to store words. BFS helps in **generating suggestions**, by traversing the tree level by level and collecting all words that start with a given prefix, sorted by proximity and frequency.

Code :

```
# Step 1: Define the graph using adjacency list (dictionary)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Step 2: BFS function using a normal Python list
def bfs(start_node):
    visited = set() # To keep track of visited nodes
    queue = [start_node] # Queue using list

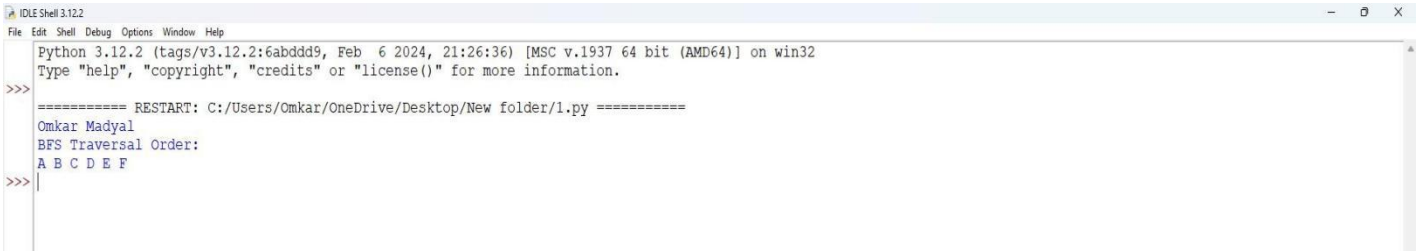
    print("BFS Traversal Order:")

    while queue:
        node = queue.pop(0) # Remove the first element (FIFO)
        if node not in visited:
            print(node, end=' ')
            visited.add(node)

            # Add unvisited neighbours to queue
            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)

# Step 3: Call BFS function
bfs('A')
```

Output :



Implement the Iterative Depth First Search algorithm to solve the same problem

Iterative Depth-First Search (DFS) is a graph or tree traversal algorithm that explores as far as possible along each branch before backtracking, but instead of using **recursion**, it uses an **explicit stack** data structure to manage traversal order. This makes the algorithm iterative in nature, as it avoids relying on the system's call stack.

In traditional DFS, recursion is commonly used to move deeper into the structure, which implicitly makes use of the call stack to remember the previous states. However, recursion can lead to problems such as **stack overflow** in deep or large graphs. The iterative approach solves this problem by replacing the function call stack with a user-defined **stack data structure**, which gives more control and safety during traversal.

The algorithm starts by pushing the **starting node** onto the stack. While the stack is not empty, the algorithm pops the top node, processes it, and then pushes all its **unvisited neighbors** onto the stack. To prevent visiting the same node multiple times (especially in graphs), a **visited set or array** is maintained.

Since the stack operates in **Last-In-First-Out (LIFO)** order, the last neighbor added to the stack is processed first. This behavior enables the algorithm to dive deep into the structure before returning to explore other branches — which is the essence of depth-first search.

In the case of **trees**, where there are no cycles, maintaining a visited array is usually unnecessary. But in **graphs**, it is essential to avoid infinite loops caused by cycles or repeated nodes.

Working Steps of Iterative DFS:

Create an empty **stack** and push the starting node onto it.

Mark the starting node as **visited**.

While the stack is not empty:

Pop the top node from the stack.

Process the node (e.g., print its value).

For each adjacent or child node of the current node:

If the node is not already visited:

Mark it as visited.

Push it onto the stack.

Time and Space Complexity:

Time Complexity: O(V + E) where V is the number of vertices and E is the number of edges. Each node and edge is processed once.

Space Complexity: O(V) in the worst case, for storing the stack and visited list.

Advantages of Iterative DFS:

Avoids function call stack overflow that occurs in recursive DFS.

Gives **more control** over traversal, especially in complex graph structures.

Can be easier to debug and modify compared to recursive implementations.

Applications:

Solving **mazes and puzzles**

Detecting **cycles** in a graph

Topological sorting in directed acyclic graphs (DAGs)

Finding **connected components** in a graph

Pathfinding and **backtracking** problems

Code :

```
def iterative_dfs(graph, start):
    visited = set()
    stack = [start]

    while stack:
        vertex = stack.pop()

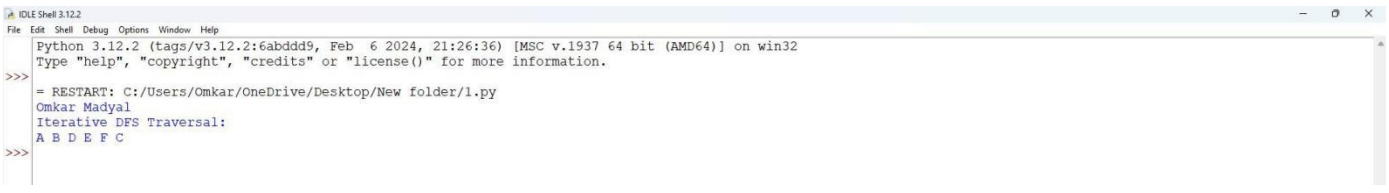
        if vertex not in visited:
            print(vertex, end=" ")
            visited.add(vertex)

            # Add unvisited neighbors to the stack
            for neighbor in reversed(graph[vertex]):
                if neighbor not in visited:
                    stack.append(neighbor)

# Example graph as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Call DFS starting from node 'A'
print("Omkar Madyal")
print("Iterative DFS Traversal:")
iterative_dfs(graph, 'A')
```

Output :



Compare the performance and efficiency of both algorithms

Both **Breadth-First Search (BFS)** and **Iterative Depth-First Search (DFS)** are fundamental algorithms used for traversing trees and graphs. While they may appear similar in terms of implementation, their **execution order, memory usage, and practical applications** differ significantly. Below is a detailed comparison based on various factors:

1. Traversal Order

BFS visits nodes **level by level**, starting from the root or source node and exploring all neighbors before moving deeper.

Iterative DFS explores nodes **as deep as possible** along each branch before backtracking, using a stack to manage the traversal path.

BFS provides a broader view; DFS dives deep into one path before exploring others.

2. Data Structure Used

BFS uses a **Queue (FIFO)** to ensure level-wise traversal.

Iterative DFS uses a **Stack (LIFO)** to simulate the recursive behavior of standard DFS.

3. Time Complexity

Both algorithms have **similar time complexity** in most cases:

BFS: O(V + E)

Iterative DFS: O(V + E)

Where:

V is the number of vertices (nodes)

E is the number of edges.

In terms of raw execution time, neither is significantly faster. Efficiency depends more on the structure of the data and the problem requirements.

4. Space Complexity

BFS: O(V) in the worst case — the queue can grow large if the tree/graph is wide.

Iterative DFS: O(V) in the worst case — the stack may grow large in deep trees or graphs.

DFS is usually more memory-efficient for deep and narrow trees. BFS may require more memory when many nodes exist at the same depth.

5. Shortest Path Finding

BFS guarantees the shortest path in an **unweighted graph**.

Iterative DFS does not guarantee the shortest path as it may find a deeper path first.

BFS is preferred for shortest path problems.

While BFS and Iterative DFS have similar time complexity, their performance and efficiency vary based on the problem context. BFS is better suited for level-based exploration and shortest-path problems, whereas Iterative DFS is ideal for deep exploration, structural analysis, and solving puzzles or backtracking-based problems. Understanding the nature of the input data and the specific requirements of the problem is essential in choosing the most appropriate traversal method.

PRACTICAL NO 2

A* Search and Recursive Best-First Search

1. A* Search

Introduction

A* (pronounced "A-star") is a widely used graph search algorithm that finds the **shortest path** from a start node to a goal node.
It is an **informed search algorithm** because it uses a **heuristic** to guide its path selection.

Key Concept

A* selects the path that minimizes the following function:

$$f(n) = g(n) + h(n)$$

Where:

- **g(n)** → Cost from the start node to the current node n
- **h(n)** → Estimated cost from n to the goal (heuristic)
- **f(n)** → Estimated total cost of the cheapest solution through n

---

Algorithm Steps

1. **Initialize**
   - Put the start node in the **open list** (priority queue) with $f(start) = h(start)$.
2. **Repeat**
   - Pick the node from the open list with the smallest $f(n)$.
   - If the node is the goal, **stop** — solution found.
   - Otherwise, expand the node and generate successors.
3. **For each successor**
   - Calculate g, h, and f values.
   - If this path is better than any previously found for the same node, update it in the open list.
4. **Continue** until the open list is empty or the goal is found.

Properties

- **Optimal**: Yes, if $h(n)$ is **admissible** (never overestimates cost to goal).
- **Complete**: Yes, if branching factor is finite and costs are positive.
- **Time complexity**: $O(b^d)$ in worst case (where b = branching factor, d = depth).
- **Space complexity**: Can be large because it keeps all generated nodes in memory.

Advantages

- Finds shortest path efficiently when heuristic is good.
- Flexible — works with different heuristics.

Limitations

- Memory-intensive.
- Performance depends heavily on the quality of the heuristic.

Example Heuristics

- **Straight-line distance** (Romania map problem)
- **Manhattan distance** (grid-based pathfinding)

Implement the A* Search algorithm for solving a pathfinding problem

DATA SET : **RMP.py** **(Save this file as RMP.py in the same folder as your main program.)**

```
print("Omkar Madyal")
import queue as Q
from RMP import dict_gn, dict_hn

start = 'Arad'
goal = 'Bucharest'
result = ''

# Function to calculate f(n) = g(n) + h(n)
def get_fn(citystr):
    cities = citystr.split(",")
    gn = 0
    for ctr in range(0, len(cities) - 1):
        gn += dict_gn[cities[ctr]][cities[ctr + 1]]
    hn = dict_hn[cities[-1]]  # heuristic from last city
```

```
    return hn + gn

# Function to expand the next city in the queue
def expand(cityq):
    global result
    if cityq.empty():
        return

    tot, citystr, thiscity = cityq.get()

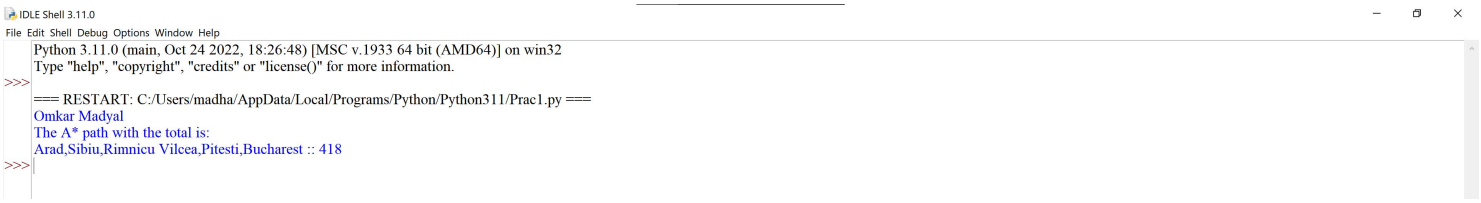
    if thiscity == goal:
        result = citystr + " :: " + str(tot)
        return

    for cty in dict_gn[thiscity]:
        cityq.put((get_fn(citystr + "," + cty), citystr + "," + cty, cty))
    expand(cityq)

# Main function
def main():
    cityq = Q.PriorityQueue()
    cityq.put((get_fn(start), start, start))
    expand(cityq)
    print("The A* path with the total is:")
    print(result)

main()
```

Output :



2. Recursive Best-First Search (RBFS)

Introduction

RBFS is a **memory-efficient version of A*** that uses recursion to explore paths.
It still uses $f(n) = g(n) + h(n)$ but stores **only limited nodes in memory**, making it suitable for large problems.

Key Idea

Instead of keeping all nodes in memory like A*, RBFS:

- Keeps track of the **best alternative path** available (the lowest f value among unexplored siblings).
- Uses recursion to explore paths one at a time.
- If the current path's $f(n)$ becomes worse than the best alternative, it **backtracks** and explores the alternative.

Algorithm Steps

1. Start with the initial node and calculate its f value.
2. Recursively expand the node with the lowest f.
3. Track a **limit** — the best f value among alternative paths.
4. If the current path's cost exceeds this limit, backtrack and follow the alternative path.
5. Continue until the goal is reached.

Properties

- **Complete**: Yes, if branching factor is finite and costs are positive.
- **Optimal**: Yes, if heuristic is admissible.
- **Space complexity**: $O(bd)$, where b = branching factor, d = depth — much better than A*.
- **Time complexity**: Can be worse than A* due to re-expanding nodes after backtracking.

Advantages

- Uses much less memory than A*.
- Suitable for large search spaces.

Limitations

- More node re-expansions (slower than A* in many cases).
- Still depends on heuristic quality.

Implement the Recursive Best-First Search algorithm for the same problem

```
print("Omkar Madyal")
import queue as Q
from RMP import dict_gn, dict_hn

start = 'Arad'
goal = 'Bucharest'
result = ''


# Function to calculate f(n) = g(n) + h(n)
def get_fn(citystr):
    cities = citystr.split(",")
    gn = 0
    for ctr in range(0, len(cities) - 1):
        gn += dict_gn[cities[ctr]][cities[ctr + 1]]
    hn = dict_hn[cities[-1]]  # heuristic from last city
    return hn + gn


# Function to print queue contents
def printout(cityq):
    for i in range(cityq.qsize()):
        print(cityq.queue[i])


# Expand the current city
def expand(cityq):
    global result

    tot, citystr, thiscity = cityq.get()
    nexttot = 999

    if not cityq.empty():
        nexttot, nextcitystr, nextthiscity = cityq.queue[0]

    if thiscity == goal and tot < nexttot:
        result = citystr + " :: " + str(tot)
        return

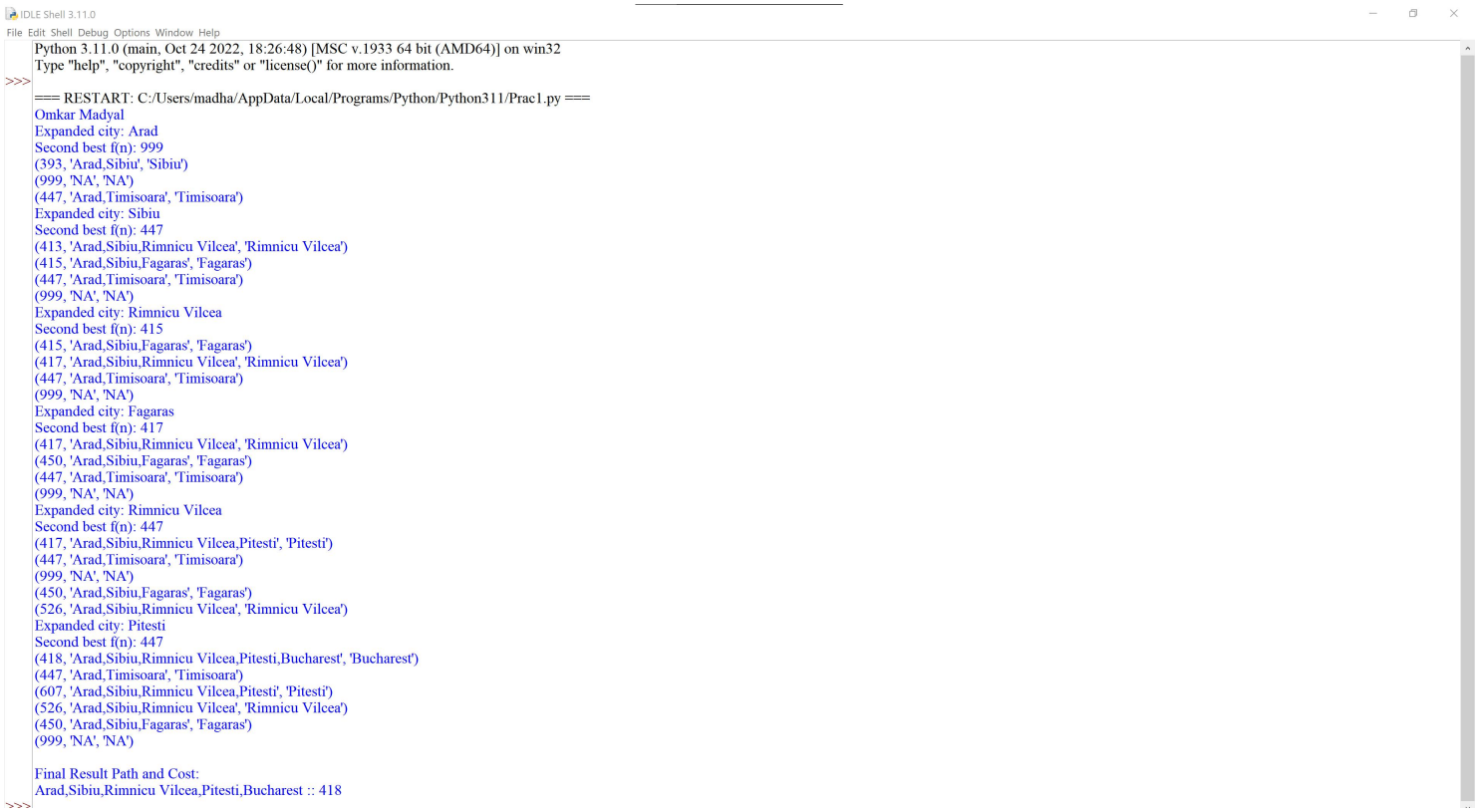
    print("Expanded city:", thiscity)
    print("Second best f(n):", nexttot)
    tempq = Q.PriorityQueue()
    for cty in dict_gn[thiscity]:
        tempq.put((get_fn(citystr + ',' + cty), citystr + ',' + cty, cty))
    for ctr in range(1, 3):  # Pick top 2 from tempq
        if not tempq.empty():
            ctrtot, ctrcitystr, ctrthiscity = tempq.get()
            if ctrtot < nexttot:
                cityq.put((ctrtot, ctrcitystr, ctrthiscity))
            else:
                cityq.put((ctrtot, citystr, thiscity))
```

```
            break

    printout(cityq)
    expand(cityq)
# Main function
def main():
    cityq = Q.PriorityQueue()
    thiscity = start
    cityq.put((999, "NA", "NA"))
    cityq.put((get_fn(start), start, thiscity))
    expand(cityq)
    print("\nFinal Result Path and Cost:")
    print(result)


main()
```

Output :



PRACTICAL NO 3

Decision Tree Learning

Implement the decision tree learning algorithm to build a decision tree for a given dataset. Evaluate the accuracy and efficiency on the test data set

1. Introduction

A **Decision Tree** is a supervised machine learning algorithm used for both **classification** and **regression** tasks.
It works by splitting the dataset into smaller subsets based on feature values, eventually resulting in a tree-like structure of decisions.

2. Structure of a Decision Tree

Root Node: Represents the entire dataset and the first decision.

Decision Nodes: Internal nodes where the dataset is split based on a feature condition.

Leaf Nodes (Terminal Nodes): Represent the final output (class label for classification or value for regression).

Branches: Arrows that connect nodes, representing the outcome of a decision.

3. Working Principle

Select the best feature to split the dataset using a criterion (e.g., Gini Index, Entropy, Information Gain).

Split the dataset into subsets based on the chosen feature.

Repeat the process recursively for each subset until:

All records in a node belong to the same class, or

There are no remaining features, or

The maximum depth is reached.

Assign the majority class (or mean value in regression) to the leaf node.

4. Splitting Criteria

For Classification Trees

Entropy & Information Gain

Entropy measures impurity:

$$Entropy = -\sum p_i \log_2(p_i)$$

- Information Gain:

$$IG = Entropy(parent) - \sum \frac{\text{Samples in child}}{\text{Samples in parent}} \times Entropy(child)$$

- Gini Index
  - Measures probability of misclassification:

$$Gini = 1 - \sum p_i^2$$

For Regression Trees

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)

5. Advantages

Easy to understand and visualize.

Handles both numerical and categorical data.

Requires little data preprocessing.

Can model non-linear relationships.

6. Disadvantages

Prone to **overfitting** (can be solved with pruning).

Small changes in data can result in a different tree (unstable).

Greedy nature may not produce the globally optimal tree.

7. Pruning

Pruning is the process of **removing branches** that have little importance to reduce overfitting.
Types:

Pre-Pruning: Stop growing the tree early.

Post-Pruning: Grow the full tree, then remove unnecessary branches.

8. Real-World Applications

Medical Diagnosis – predicting disease based on symptoms.

Finance – credit risk assessment.

Marketing – customer segmentation.

Manufacturing – defect detection

Code :

```
print("Omkar Madyal")
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import time

# 1. Load the dataset from an online Gist (PlayTennis.csv)
url = "https://gist.githubusercontent.com/DiogoRibeiro7/c6590d0cf119e87c39e31c21a9c0f3a8/raw/PlayTennis.csv"
data = pd.read_csv(url)

# 2. Preprocess: one-hot encode categorical features, encode target
X = pd.get_dummies(data.drop(columns=['Play Tennis']))
y = data['Play Tennis'].map({'Yes': 1, 'No': 0})

# 3. Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 4. Train the Decision Tree classifier (using entropy / ID3-style splitting)
start_train = time.time()
clf = DecisionTreeClassifier(criterion='entropy')
clf.fit(X_train, y_train)
train_time = time.time() - start_train

# 5. Predict on test set and measure prediction time
start_pred = time.time()
y_pred = clf.predict(X_test)
pred_time = time.time() - start_pred

# 6. Evaluate accuracy
accuracy = accuracy_score(y_test, y_pred)

# 7. Display decision rules and timing metrics
print("Learned Decision Tree Rules:\n")
print(export_text(clf, feature_names=list(X.columns)))
print(f"\nAccuracy: {accuracy:.2f}")

print(f"Training Time: {train_time:.6f} seconds")
```

```
print(f"Prediction Time: {pred_time:.6f} seconds")

# 8. Plot the decision tree structure
plt.figure(figsize=(12, 8))
plot_tree(
    clf,
    feature_names=X.columns,
    class_names=['No', 'Yes'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title("Decision Tree for Play Tennis Dataset")
plt.show()
```

IDLE Shell 3.11.0

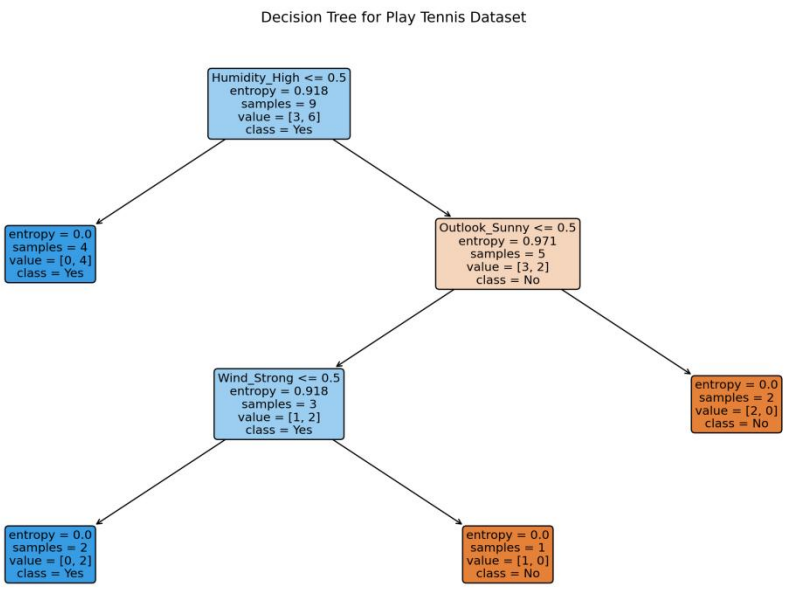
File Edit Shell Debug Options Window Help

```
Python 3.11.0 (main, Oct 24 2022, 18:26:48) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
=== RESTART: C:/Users/madha/AppData/Local/Programs/Python/Python311/Prac1.py ===
Omkar Madyal
Learned Decision Tree Rules:

|--- Humidity_High <= 0.50
|   |--- class: 1
|--- Humidity_High >  0.50
|   |--- Outlook_Sunny <= 0.50
|   |   |--- Wind_Strong <= 0.50
|   |   |   |--- class: 1
|   |   |--- Wind_Strong >  0.50
|   |   |   |--- class: 0
|   |--- Outlook_Sunny >  0.50
|   |   |--- class: 0


Accuracy: 0.60
Training Time: 0.003000 seconds
Prediction Time: 0.000999 seconds
```

Figure 1



PRACTICAL NO 4

Feed Forward Back propagation Neural Network

- Implement the Feed Forward Back propagation algorithm to train a neural network
- Use a given dataset to train the neural network for a specific task

Feed Forward Back Propagation Neural Network (FF-BPNN)

1. Introduction

A **Feed Forward Back Propagation Neural Network** is a type of **Artificial Neural Network (ANN)** where:

- Data flows **forward** from input → hidden layers → output.
- Errors are propagated **backward** during training to adjust weights using the **Back Propagation** algorithm.

It is widely used for:

- Classification
- Regression
- Pattern recognition
- Time-series prediction

---

2. Structure

A typical FF-BPNN consists of:

1. **Input Layer**
   - Receives the input data.
   - Number of neurons = number of features.
2. **Hidden Layer(s)**
   - Perform intermediate computations.
   - Each neuron applies a **weighted sum + activation function**.
3. **Output Layer**
   - Produces the final result (class label, probability, etc.).
   - Activation depends on problem type (e.g., **Sigmoid** for binary classification, **Softmax** for multi-class).

---

3. Feed Forward Process

1. **Weighted Sum**

$$z = \sum(w_i \cdot x_i) + b$$

where:

- w_i = weight
- x_i = input
- b = bias

2. **Activation Function**

$$a = f(z)$$

   Common functions: **Sigmoid**, **Tanh**, **ReLU**.

3. **Layer-to-Layer Transmission**
   The output from one layer becomes the input to the next layer until the final output layer is reached.

---

4. Back Propagation (Training)

The goal is to **minimize error** between predicted output (y_{pred}) and actual output (y_{true}).

Steps:

1. **Calculate Error**
   For example, Mean Squared Error (MSE):

$$E = \frac{1}{2}\sum(y_{true} - y_{pred})^2$$

2. **Backward Pass**
   - Compute the gradient of the error w.r.t each weight using **Chain Rule**.
   - Update weights using **Gradient Descent**:

$$w_{new} = w_{old} - \eta \cdot \frac{\partial E}{\partial w}$$

     where η = learning rate.

3. **Repeat**
   Until convergence (error is minimal).

---

5. Gini Impurity Concept

Although **Gini Impurity** is a measure used in **Decision Trees**, it can conceptually be connected to neural networks during classification:

- **Gini Impurity Formula**:

$$G = 1 - \sum_{i=1}^{n} p_i^2$$

  where:

  - p_i = probability of class i in a node.
  - Lower Gini means **purer** data (less mixed classes).

Relation to FF-BPNN:

In a decision tree, Gini Impurity helps **split data** to make pure subsets.

In FF-BPNN, although we do not explicitly calculate Gini, the **loss function** (e.g., Cross-Entropy) plays a similar role of ensuring correct class separation.

You can combine them in hybrid models: first **split data with decision tree (Gini)**, then use **FF-BPNN** for final classification.

6. Advantages

Can approximate any non-linear function.

Works well for large and complex datasets.

Learns internal feature representation.

7. Limitations

Requires large amounts of data for training.

Prone to overfitting.

Hyperparameter tuning is critical.

8. Applications

Image recognition

Speech processing

Financial prediction

Medical diagnosis

Code :

```
print("Omkar Madyal")
import numpy as np

class NeuralNetwork:
    def __init__(self):
        np.random.seed(1)  # For reproducibility
        # Randomly initialize weights with mean 0
        self.synaptic_weights = 2 * np.random.random((3, 1)) - 1

    def sigmoid(self, x):
        """Sigmoid activation function"""
        return 1 / (1 + np.exp(-x))
```

```
    def sigmoid_derivative(self, x):
        """Derivative of sigmoid function"""
        return x * (1 - x)

    def train(self, training_inputs, training_outputs, training_iterations):
        """Train the neural network using backpropagation"""
        for iteration in range(training_iterations):
            # Forward pass
            output = self.think(training_inputs)

            # Calculate the error
            error = training_outputs - output

            # Backpropagation
            adjustments = np.dot(training_inputs.T, error * self.sigmoid_derivative(output))

            # Adjust weights
            self.synaptic_weights += adjustments

    def think(self, inputs):
        """Pass inputs through the network"""
        inputs = inputs.astype(float)
        output = self.sigmoid(np.dot(inputs, self.synaptic_weights))
        return output


if __name__ == "__main__":
    # Initializing the neuron class
    neural_network = NeuralNetwork()

    print("Beginning Randomly Generated Weights:")
    print(neural_network.synaptic_weights)

    # Training data: 4 examples, 3 input values each, 1 output value
    training_inputs = np.array([
        [0, 0, 1],
        [1, 1, 1],
        [1, 0, 1],
        [0, 1, 1]
    ])

    training_outputs = np.array([[0, 1, 1, 0]]).T

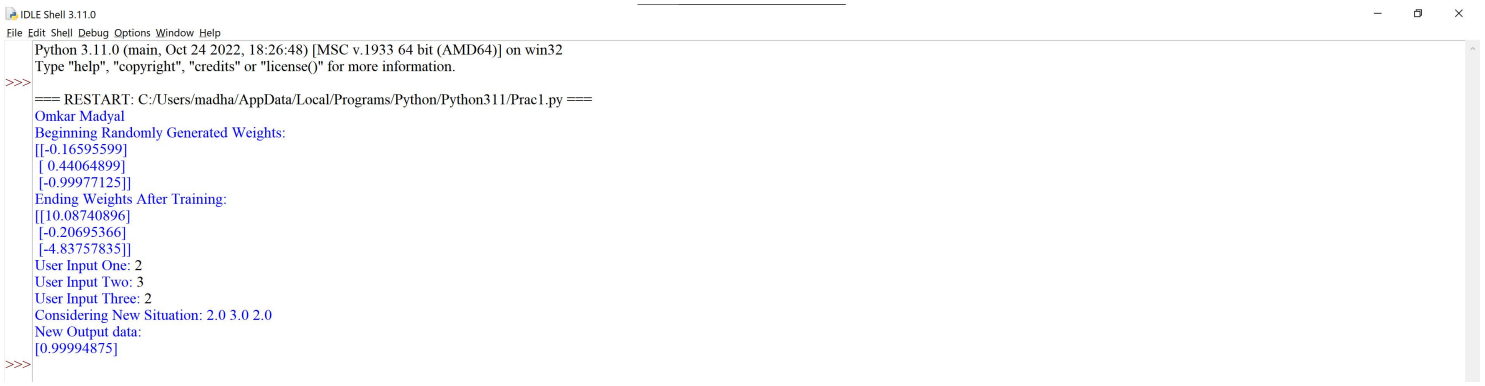
    # Training the neural network
    neural_network.train(training_inputs, training_outputs, 15000)

    print("Ending Weights After Training:")
    print(neural_network.synaptic_weights)

    # Testing with new input
    user_input_one = float(input("User Input One: "))
    user_input_two = float(input("User Input Two: "))
    user_input_three = float(input("User Input Three: "))
```

```
print("Considering New Situation:", user_input_one, user_input_two, user_input_three)
print("New Output data:")
print(neural_network.think(np.array([user_input_one, user_input_two, user_input_three])))
```

Output :



PRACTICAL NO 5

Support Vector Machines (SVM)

Implement the SVM algorithm for binary classification. Train a SVM Model using the given dataset. Evaluate the performance on test data and analyze the results

1. Introduction

Support Vector Machine (SVM) is a **supervised machine learning algorithm** used for:

- **Classification** (main application)
- **Regression** (less common, called SVR)

SVM aims to **find the best boundary (hyperplane)** that separates data points of different classes.

---

2. The Main Idea

- Imagine plotting data points in space.
- SVM tries to find a **decision boundary** (line in 2D, plane in 3D, hyperplane in higher dimensions) that **maximizes the margin** between the classes.
- **Margin**: The distance between the hyperplane and the nearest data points from each class.
- These nearest points are called **Support Vectors**.

---

3. Why Maximize the Margin?

- A larger margin means better generalization to unseen data.
- SVM chooses the hyperplane that has the **largest possible margin** and still separates the classes correctly.

---

4. Mathematically

For a binary classification:
Let:

- Training data: (x_i, y_i) where $y_i \in \{-1, +1\}$
- The hyperplane equation:

$$w \cdot x + b = 0$$

Where:

- w = weight vector (defines direction of the hyperplane)
- b = bias term (shifts the hyperplane)

The decision rule is:

$$\text{If } w \cdot x + b \geq 0 \Rightarrow \text{Class } +1$$

$$\text{If } w \cdot x + b < 0 \Rightarrow \text{Class -1}$$

5. The Optimization Goal

SVM solves:

$$\text{Minimize: } \frac{1}{2}\|w\|^2$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1$$

This ensures:

- Maximum margin
- Correct classification

---

6. Handling Non-Separable Data

Real-world data isn't always perfectly separable.

SVM uses:

- **Soft Margin** → Allows some misclassification with a penalty (C parameter controls it).
- **Kernels** → Map data to higher dimensions where it becomes separable.

7. Kernel Trick

Instead of explicitly mapping data into higher dimensions, SVM uses **kernel functions** to compute the dot product in that space efficiently.

Common kernels:

- **Linear Kernel**: Good for linearly separable data.
- **Polynomial Kernel**: Captures polynomial relationships.
- **RBF (Radial Basis Function)**: Works well for most non-linear problems.
- **Sigmoid Kernel**: Similar to neural networks.

---

8. Advantages of SVM

- Works well in high-dimensional spaces.
- Effective when number of dimensions > number of samples.
- Robust to overfitting if parameters are well chosen.

---

9. Disadvantages

- Can be slow for large datasets.
- Performance depends on the right choice of kernel & parameters.
- Less effective for noisy data.

10. Applications

- Text classification (spam detection)
- Face recognition
- Bioinformatics (gene classification)
- Handwriting recognition

Quick Diagram in Words

- **Support Vectors** → Points on the edge of the margin
- **Hyperplane** → Decision boundary
- **Margin** → Space between support vectors of different classes

Code:

```
print("Omkar Madyal")
# Import required libraries
from warnings import filterwarnings
filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
import time

# ==================================
# Step 1: Load the Dataset Online
# ==================================

# URL of the dataset hosted on GitHub
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"

# Define column names manually (since dataset has no header row)
column_names = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
                'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome']

# Load the dataset
df = pd.read_csv(url, names=column_names)

# Show first few rows
print("First 5 rows of the dataset:")
print(df.head())
```

```
# Dataset shape and stats
print("\nDataset shape:", df.shape)
print("\nStatistical Summary:")
print(df.describe())

# ============================
# Step 2: Prepare the Data
# ============================

# Features and target
X = df.drop("Outcome", axis=1) # Independent variables
y = df["Outcome"] # Target variable

# Manual train-test split (600 train + 168 test)
X_train = X.iloc[:600]
X_test = X.iloc[600:]
y_train = y.iloc[:600]
y_test = y.iloc[600:]

print("\nTraining set shape:", X_train.shape)
print("Test set shape:", X_test.shape)

# ================================
# Step 3: Train SVM with Linear Kernel
# ================================

model = SVC(kernel='linear') # Linear SVM
model.fit(X_train, y_train) # Train the model

# ============================
# Step 4: Predictions & Metrics
# ============================

y_pred = model.predict(X_test)

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Classification report
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# ================================
# Step 5: Cross-Validation
# ================================

# 10-fold cross-validation on training set
cv_scores = cross_val_score(model, X_train, y_train, cv=10)

print("\nCross-Validation Accuracy: {:.2f}%".format(cv_scores.mean() * 100))
```

```
print("Standard Deviation: {:.2f}%".format(cv_scores.std() * 100))

# ==================================
# Step 6: Hyperparameter Tuning
# ==================================

# Define values of C to try
param_grid = {'C': np.arange(1, 20)} # Regularization parameter

# GridSearch with 8-fold CV
grid_search = GridSearchCV(SVC(kernel='linear'), param_grid, cv=8)

start_time = time.time()
grid_search.fit(X_train, y_train)
elapsed_time = time.time() - start_time

print("\nGrid Search completed in {:.2f} seconds".format(elapsed_time))
print("Best Parameters:", grid_search.best_params_)
print("Best CV Score: {:.2f}%".format(grid_search.best_score_ * 100))

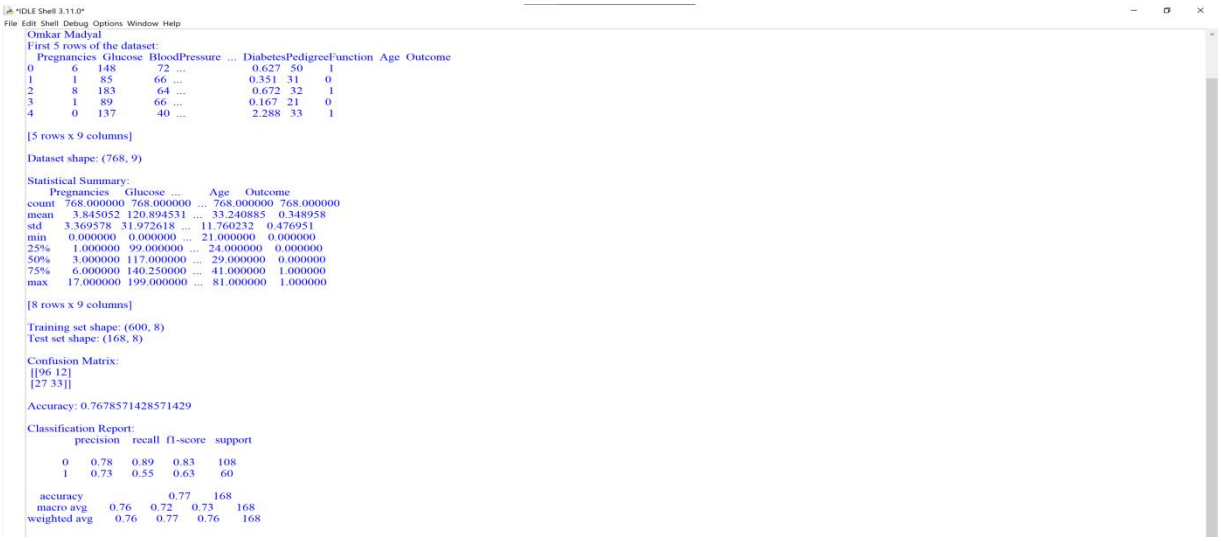
# ==================================
# Step 7: Final Model Evaluation
# ==================================

# Train final model using best parameters
final_model = SVC(kernel='linear', C=grid_search.best_params_['C'])
final_model.fit(X_train, y_train)

# Predict and evaluate again
final_pred = final_model.predict(X_test)
final_cm = confusion_matrix(y_test, final_pred)

print("\nFinal Confusion Matrix:\n", final_cm)
print("Final Accuracy: {:.2f}%".format(accuracy_score(y_test, final_pred) * 100))
print("Final Classification Report:\n", classification_report(y_test, final_pred))
```

Output :



PRACTICAL NO 6

Adaboost Ensemble Learning

- Implement the Adaboost algorithm to create an ensemble of weak classifiers.
- Train the ensemble model on a given dataset and evaluate its performance
- Compare the results with individual weak classifiers

1. Introduction

AdaBoost, short for **Adaptive Boosting**, is a powerful ensemble learning technique introduced by **Yoav Freund and Robert Schapire in 1996**. It is one of the first boosting algorithms that gained popularity due to its simplicity and effectiveness. AdaBoost combines multiple **weak learners** (usually decision stumps) to form a **strong learner** with high predictive accuracy.

2. Concept of Ensemble Learning

Ensemble learning is based on the principle that combining the predictions of multiple models can result in better performance than any individual model. There are three main types of ensemble methods:

Bagging (Bootstrap Aggregating) – e.g., Random Forest

Boosting – e.g., AdaBoost, Gradient Boosting, XGBoost

Stacking – combining predictions of different model types

AdaBoost falls under the **Boosting** category, where weak learners are trained sequentially, and each new learner focuses more on the mistakes made by the previous ones.

3. Idea Behind AdaBoost

The idea of AdaBoost is to:

Train multiple weak learners one after another.

Assign **weights** to each training sample.

In each iteration, **increase the weight** of misclassified samples so that the next model pays more attention to them.

Combine all weak learners using a weighted majority vote (for classification) or weighted average (for regression).

4. Working of AdaBoost (Step-by-Step)

Let's assume we are solving a **binary classification problem**.

Step 1: Initialize Weights

Initially, all training samples are assigned equal weights:
wi=1/ N where N is the number of samples.

Step 2: Train a Weak Learner

A weak learner is trained on the weighted dataset.

Commonly used weak learner: **Decision Stump** (a one-level decision tree).

Step 3: Calculate Error

Weighted error is calculated as:

$$\text{Error} = \frac{\sum_{i=1}^{N} w_i \cdot I(y_i \neq h_t(x_i))}{\sum_{i=1}^{N} w_i}$$

where:

- I is the indicator function (1 if misclassified, else 0)
- $h_t(x)$ is the prediction from weak learner t

Step 4: Compute Learner Weight

- The importance (weight) of this weak learner is calculated as:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \text{Error}}{\text{Error}} \right)$$

  - A learner with low error gets a higher weight.

Step 5: Update Sample Weights

- Increase the weights of misclassified samples and decrease the weights of correctly classified ones:

$$w_i \leftarrow w_i \cdot e^{\alpha_t \cdot I(y_i \neq h_t(x_i))}$$

- Normalize the weights so that they sum to 1.

Step 6: Repeat

- Repeat Steps 2–5 for a predefined number of iterations (n_estimators) or until error is minimal.

Step 7: Final Prediction

- For classification:

$$H(x) = \text{sign} \left(\sum_{t=1}^{T} \alpha_t \cdot h_t(x) \right)$$

- For regression: a weighted sum of learners' predictions.

---

5. Key Characteristics

Adaptive: Focuses more on difficult samples by adjusting weights.

Sequential: Each weak learner depends on the outcome of the previous one.

Weighted Voting: Not all learners contribute equally; stronger learners have more influence.

6. Advantages

Simple to implement.

Often more accurate than a single model.

Less prone to overfitting compared to other methods if weak learners are simple.

Works well with both classification and regression.

7. Limitations

Sensitive to noisy data and outliers (as weights of misclassified samples increase).

Computationally slower than bagging methods for large datasets.

Weak learners must perform slightly better than random guessing; otherwise, boosting can fail.

8. Applications

Spam email classification

Fraud detection in banking

Medical diagnosis prediction

Face detection in computer vision

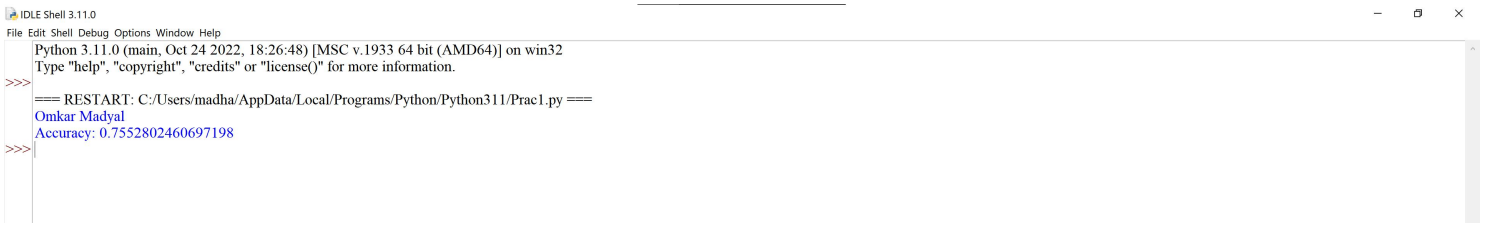
Sentiment analysis

Code:

```
print("Omkar Madyal")
import pandas as pd
from sklearn import model_selection
from sklearn.ensemble import AdaBoostClassifier
# Correct dataset URL
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
# Column names
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']
# Load dataset
```

```
dataframe = pd.read_csv(url, names=names)
# Separate into input and output
array = dataframe.values
X = array[:, 0:8]
Y = array[:, 8]
# Parameters
seed = 7
num_trees = 30
# Define kfold cross-validation
kfold = model_selection.KFold(n_splits=10, shuffle=True, random_state=seed)
# Define model
model = AdaBoostClassifier(n_estimators=num_trees, random_state=seed)
# Evaluate model
results = model_selection.cross_val_score(model, X, Y, cv=kfold)
print("Accuracy:", results.mean())
```

Output



PRACTICAL NO 7

Naive Bayes' Classifier

- Implement the Naive Bayes algorithm for classification
- Trin a Naive Bayes' model using a given dataset and calculate class probabilities
- Evaluate the accuracy of the model on test data and analyze the results

Naïve Bayes Classifier is a **probabilistic machine learning algorithm** based on **Bayes' Theorem**.It is called *"naïve"* because it assumes that all features (symptoms, words, attributes, etc.) are **independent of each other**, which is rarely true in real life but works surprisingly well.

Commonly used for:
Text classification (spam detection, sentiment analysis)
Medical diagnosis (symptom → disease)
Document categorization

2. Bayes' Theorem

Bayes' Theorem states:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where:

- C = class (example: Disease = "Flu")
- X = features (example: Fever, Cough, Headache)
- $P(C|X)$ = Posterior probability (probability of class C given features X)
- $P(C)$ = Prior probability of class (how common the disease is)
- $P(X|C)$ = Likelihood (probability of seeing those symptoms given the disease)
- $P(X)$ = Evidence (probability of seeing those symptoms overall)

---

3. Naïve Independence Assumption

The algorithm assumes that all features are **conditionally independent** given the class.

So instead of calculating $P(X|C)$ directly (which is complex), we use:

$$P(X|C) = P(x_1|C) \cdot P(x_2|C) \cdot ... \cdot P(x_n|C)$$

Where:

- $x_1, x_2, ..., x_n$ are individual features (e.g., Fever, Cough).

This simplification makes computations **fast and feasible**.

---

4. Working of Naïve Bayes Classifier

1. **Training Phase**:
   - Calculate the **prior probability** of each class:

$$P(C) = \frac{\text{Number of samples in class C}}{\text{Total samples}}$$

   - Calculate the **conditional probability** for each feature given the class:

$$P(x_i|C) = \frac{\text{Number of times feature } x_i \text{ appears in class C}}{\text{Total samples in class C}}$$

2. **Prediction Phase**:
   - For a new input X, compute the posterior probability for each class:

$$P(C|X) \propto P(C) \cdot \prod_i P(x_i|C)$$

   - Choose the class with the **highest probability**.

Types of Naïve Bayes Classifiers

Gaussian Naïve Bayes

Assumes features are **continuous** and follow a **Gaussian (Normal) distribution**.

Example: Medical test results, height/weight data.

Multinomial Naïve Bayes

Works with **discrete count data** (word frequencies in documents).

Example: Text classification, spam filtering.

Categorical/Bernoulli Naïve Bayes

Works with **binary/categorical features** (yes/no, 0/1).

Example: Symptom present or not

Code:

```
print('Omkar Madyal')
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB, CategoricalNB, GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
from sklearn.preprocessing import LabelEncoder

# Load dataset from URL (symptoms vs disease dataset)
url = "https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv"
df = pd.read_csv(url, on_bad_lines="skip", engine="python")

print("Dataset preview:")
print(df.head())

# For this dataset, target column = "Outcome"
X = df.drop(columns=["Outcome"])
y = df["Outcome"]

# Encode target if needed
if y.dtype == "object":
    y = LabelEncoder().fit_transform(y)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Try different Naive Bayes classifiers
```

```
models = {
    "GaussianNB": GaussianNB(),
    "MultinomialNB": MultinomialNB(),
    "CategoricalNB": CategoricalNB()
}

for name, model in models.items():
    try:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        print(f"\n====== {name} ======")
        print("Accuracy:", accuracy_score(y_test, y_pred))
        print(classification_report(y_test, y_pred))

        # Confusion matrix heatmap
        cm = confusion_matrix(y_test, y_pred)
        plt.figure(figsize=(6, 4))
        sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
        plt.title(f"Confusion Matrix - {name}")
        plt.xlabel("Predicted")
        plt.ylabel("Actual")
        plt.show()
    except Exception as e:
        print(f"\n{name} failed: {e}")
```

Output :

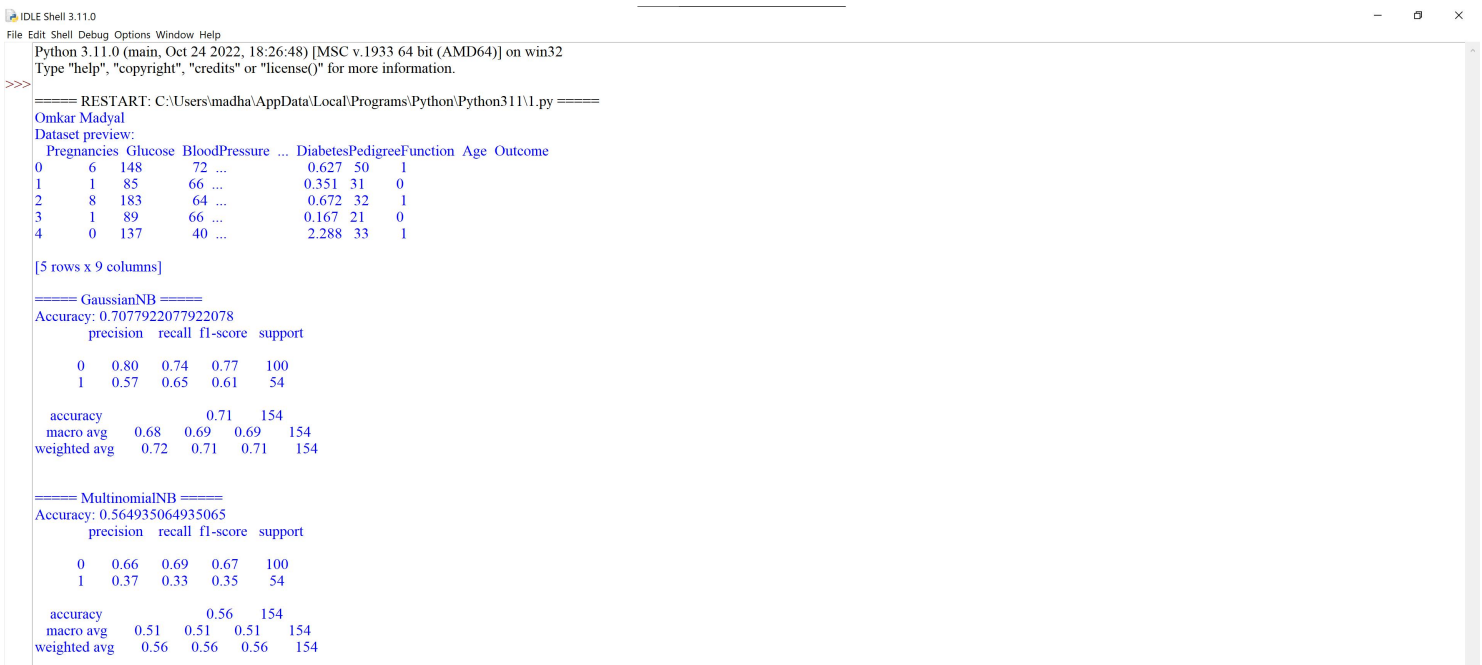


Figure 1

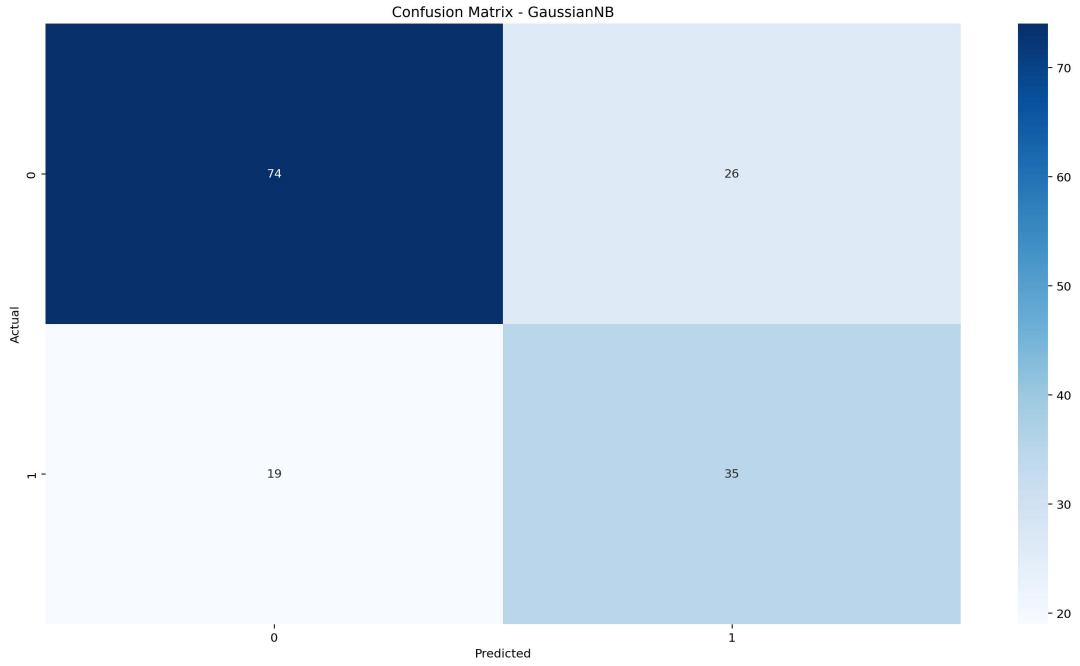
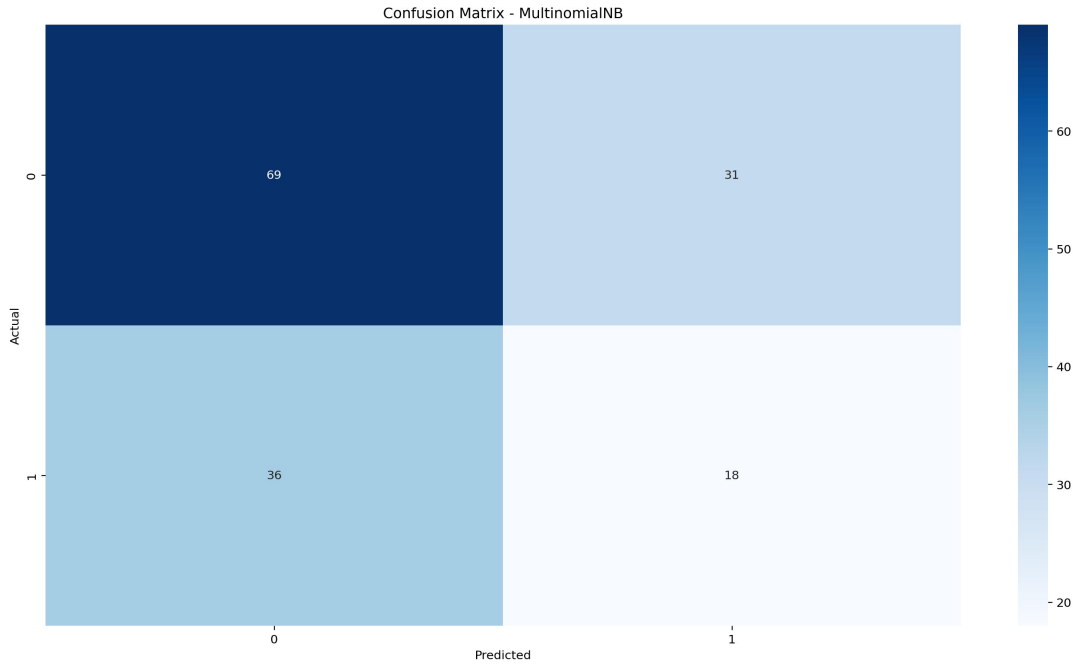


Figure 1



PRACTICAL NO 8

Implement the K-NN Algorithm for classification or regression

K-**Nearest Neighbors (KNN)** is a **supervised machine learning algorithm** used for **classification** and **regression**.It is a **lazy learner** (no explicit training phase). Instead, it memorizes the dataset and makes predictions based on the nearest neighbors at query time.It is a **non-parametric algorithm**, meaning it makes no assumptions about the underlying data distribution.

Basic Idea

To classify a new data point, the algorithm:
Finds the K nearest data points in the training set (using distance measures like Euclidean distance).
Looks at the labels of those K neighbors.
Assigns the label that is most frequent among them (majority voting).
For regression, it takes the average (or weighted average) of the K neighbors' values.

3. Distance Metrics

KNN uses a **distance function** to find the nearest neighbors. Common ones are:

- **Euclidean Distance** (most common):

$$d(p,q) = \sqrt{\sum_{i=1}^{n}(p_i - q_i)^2}$$

- **Manhattan Distance**:

$$d(p,q) = \sum_{i=1}^{n}|p_i - q_i|$$

- **Minkowski Distance** (generalized form).
- **Cosine Similarity** (for text/data in high-dimensional space).

---

4. Choice of K

- **Small K (e.g., K=1)** → Model becomes sensitive to noise, risk of overfitting.
- **Large K** → Smoother decision boundary, but may ignore local patterns (risk of underfitting).
- Usually, K is chosen as an **odd number** (to avoid ties in binary classification).

---

5. Working of KNN (Step-by-Step)

1. **Choose K** (number of neighbors).
2. **Compute distance** between the new point and all training points.
3. **Select K nearest neighbors**.
4. **Classification** → Majority class wins.
   Regression → Average value of neighbors.

---

6. Properties of KNN

- **Lazy learning**: No training, only computation during prediction.
- **Instance-based learning**: Uses the entire training set to make predictions.
- **Decision boundaries**: Non-linear, adapt to data distribution.

- Apply K-NN Algorithm on the given dataset & predict the class or value for test data

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Use ggplot style
plt.style.use('ggplot')

# Load dataset
url = "https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv"
df = pd.read_csv(url)

print("Dataset preview:")
print(df.head())

# Features and target
X = df.drop(columns=["Outcome"])
y = df["Outcome"]

# Scale features (important for distance-based models like KNN)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# Try KNN with different values of k
k_values = range(1, 21)
accuracies = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)

# Plot accuracy vs K
plt.figure(figsize=(8, 5))
plt.plot(k_values, accuracies, marker='o', linestyle='-', color='b')
plt.title("KNN Accuracy for Different K Values")
plt.xlabel("Number of Neighbors (K)")
plt.ylabel("Accuracy")
plt.xticks(k_values)
```

```
plt.show()

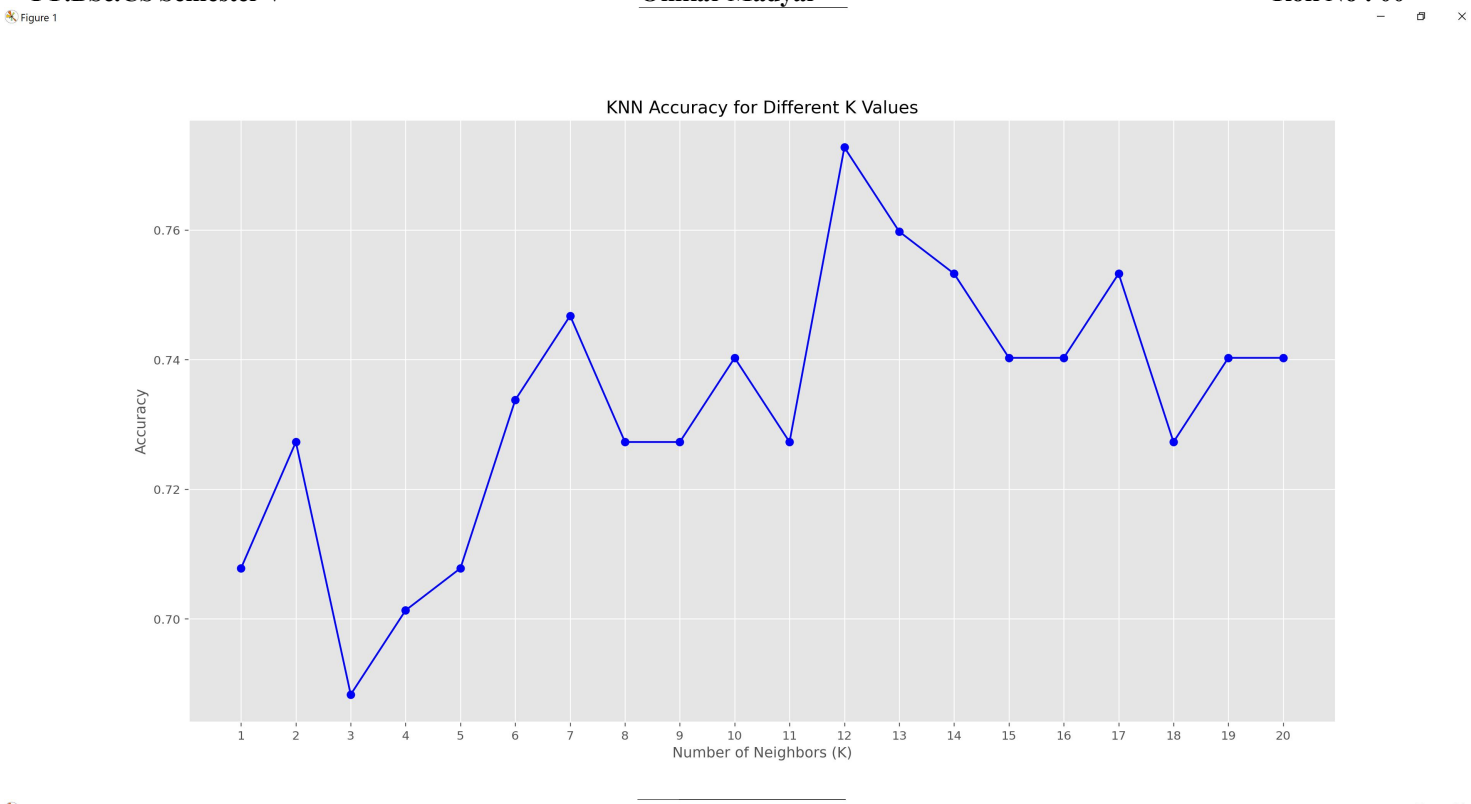
# Best K
best_k = k_values[np.argmax(accuracies)]
print(f"Best K: {best_k} with Accuracy: {max(accuracies):.4f}")

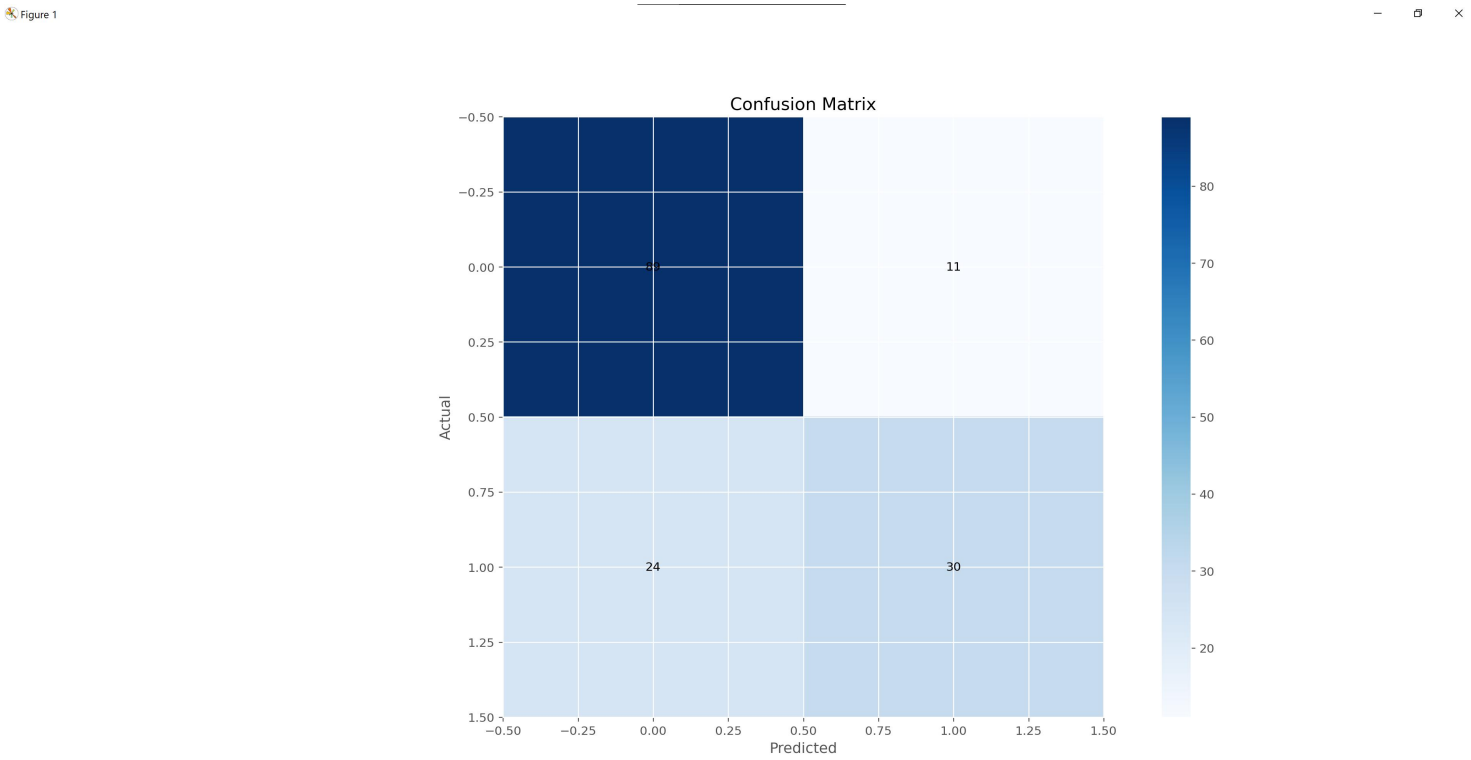
# Final model with best K
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, y_train)
y_pred = knn_best.predict(X_test)

# Evaluation
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5, 4))
plt.imshow(cm, cmap="Blues")
plt.colorbar()
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, cm[i, j], ha='center', va='center', color='black')
plt.show()
```




Figure 1
KNN Accuracy for Different K Values
Accuracy
Number of Neighbors (K)
0.76
0.74
0.72
0.70
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20


Figure 1
Confusion Matrix
Actual
Predicted
11
24
30
−0.50 −0.25 0.00 0.25 0.50 0.75 1.00 1.25 1.50
80 70 60 50 40 30 20

PRACTICAL NO 9

Implement the Association Rule Mining algorithm (e.g. Apriori) to find frequent dataset. Generate association rules from the frequent item set and calculate their support.

1. Definition

Association Rule Mining is a data mining technique used to discover interesting relationships, patterns, or correlations among items in large datasets.

It is widely applied in **Market Basket Analysis**, where retailers want to find rules like:

- *"If a customer buys bread and butter, they are likely to buy milk too."*

---

2. Key Concepts

1. **Itemset**

   A collection of one or more items.

   Example: `{bread, butter}`, `{milk}`

2. **Support**

   Probability that a transaction contains an itemset.

$$Support(X) = \frac{\text{No. of transactions containing X}}{\text{Total transactions}}$$

3. **Confidence**

   Likelihood of buying Y when X is bought.

$$Confidence(X \Rightarrow Y) = \frac{Support(X \cup Y)}{Support(X)}$$

4. **Lift**

   Measure of how much more likely X and Y occur together than expected if independent.

$$Lift(X \Rightarrow Y) = \frac{Support(X \cup Y)}{Support(X) \times Support(Y)}$$

   - Lift > 1 → Positive association
   - Lift = 1 → Independent
   - Lift < 1 → Negative association

3. Idea

Apriori uses the principle:

👉 *"If an itemset is frequent, all of its subsets must also be frequent."*

👉 *"If an itemset is infrequent, all its supersets are infrequent."*

This **Apriori Property** reduces the search space.

---

4. Steps of Apriori Algorithm

1. **Input**:
   - Transaction Dataset
   - Minimum Support Threshold
   - Minimum Confidence Threshold
2. **Process**:
   - **Step 1**: Generate **frequent 1-itemsets** (items appearing above min support).
   - **Step 2**: Generate **candidate k-itemsets** by joining frequent (k−1)-itemsets.
   - **Step 3**: Prune candidates that contain infrequent subsets (Apriori property).
   - **Step 4**: Count support of candidates using dataset scan.
   - **Step 5**: Keep only those above min support → **frequent itemsets**.
   - **Step 6**: Repeat until no more frequent itemsets can be formed.
   - **Step 7**: Generate **association rules** from frequent itemsets and filter by min confidence.
3. **Output**:
   Strong association rules of the form:

$$X \Rightarrow Y$$

---

5. Example

Dataset of 5 transactions:

TID	Items
T1	{Milk, Bread, Butter}
T2	{Bread, Butter}
T3	{Milk, Bread}
T4	{Milk, Bread, Butter}
T5	{Milk, Butter}

- **Support({Milk, Bread}) = 3/5 = 0.6**
- **Confidence({Milk} ⇒ {Bread}) = Support({Milk, Bread}) / Support({Milk}) = 0.6 / 0.8 = 0.75**

Code:

```
import numpy as np
import pandas as pd
import plotly.graph_objects as go
```

```
import plotly.express as px
from apyori import apriori
df=pd.read_csv('Groceries_dataset.csv')
df.head()
df.isnull().any()
all_products=df['itemDescription'].unique()
print("Total Products :{}".format(len(all_products)))
def distribution_plot(x,y,name=None,xaxis=None,yaxis=None):
  fig=go.Figure([go.Bar(x=x,y=y)])
  fig.update_layout(title=name,xaxis_title=xaxis,yaxis_title=yaxis)
  fig.show()

x=df['itemDescription'].value_counts()
x=x.sort_values(ascending=False)
x=x[:10]

distribution_plot(x=x.index,y=x.values,yaxis="Count",xaxis="Products")

one_hot=pd.get_dummies(df['itemDescription'])
df.drop('itemDescription',inplace=True,axis=1)
df=df.join(one_hot)
df.head()
records=df.groupby(['Member_number','Date'])[all_products[:]].apply(sum)
records=records.reset_index()[all_products]
records.head()
records.shape

def get_Pnames(x):
  for product in all_products:
    if x[product]>0:
      x[product]=product
  return x

records=records.apply(get_Pnames,axis=1)
records.head()
records.shape
print("Total Transactions:{}".format(len(records)))
x=records.values
x=[sub[~(sub == 0)].tolist() for sub in x if sub[sub != 0].tolist()]
transactions=x
rules=apriori(transactions,min_support=0.00030,min_confidence=0.2,min_lift=3,min_length=2)
association_results=list(rules)

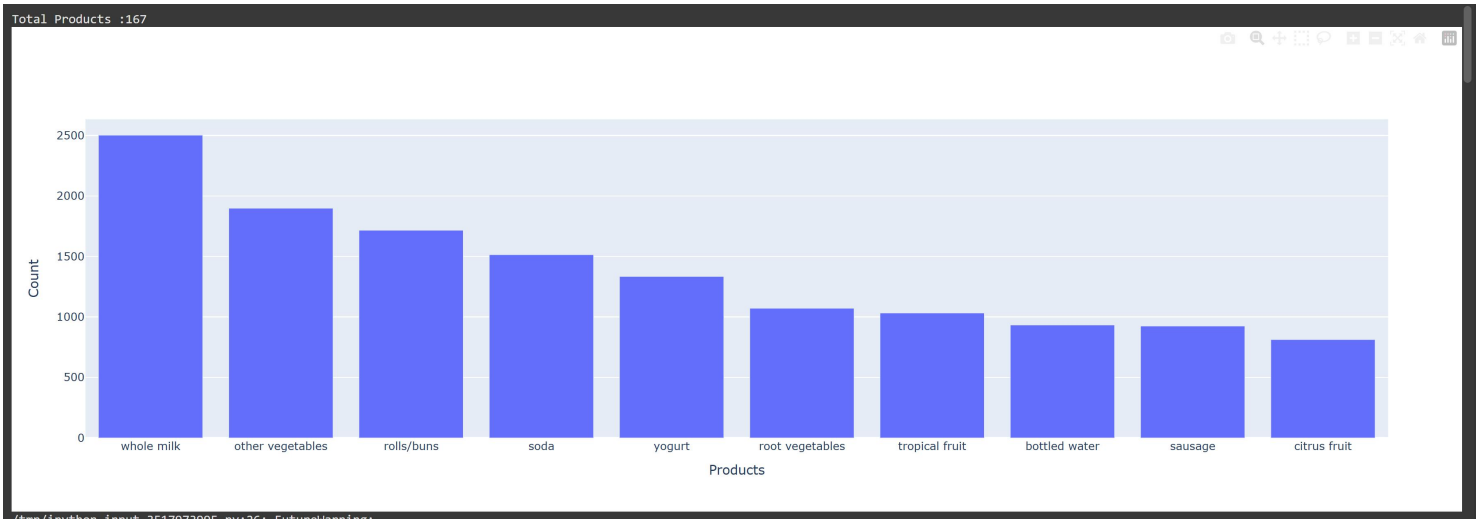
for item in association_results:
  pair=item[0]
  items=[x for x in pair]
  print("Rule:"+items[0]+"->"+items[1])
  print("Support:"+str(item[1]))
  print("Confidence:"+str(item[2][0][2]))
  print("Lift:"+str(item[2][0][3]))
  print("=====================================")
```

Output :

1 to 5 of 5 entries Filter

index	Member_number	Date	itemDescription
0	1808	21-07-2015	tropical fruit
1	2552	05-01-2015	whole milk
2	2300	19-09-2015	pip fruit
3	1187	12-12-2015	other vegetables
4	3037	01-02-2015	whole milk

```
Total Products :167
```




```
Total Transactions:14963
Rule:butter->sausage
Support:0.0003341575887188398
Confidence:0.2
Lift:3.3140642303433
=====================================
Rule:brown bread->frozen vegetables
Support:0.0003341575887188398
Confidence:0.35714285714285715
Lift:7.612433862433862
=====================================
Rule:brown bread->sausage
Support:0.0004009891064626077 5
Confidence:0.2222222222222222
Lift:4.736625514403292
=====================================
Rule:yogurt->canned beer
Support:0.0003341575887188398
Confidence:0.4166666666666667
Lift:4.851815823605707
=====================================
Rule:frozen vegetables->sausage
Support:0.0003341575887188398
Confidence:0.25
Lift:4.142580287929125
=====================================
Rule:whole milk->hygiene articles
Support:0.0004009891064626077 5
Confidence:0.5454545454545454
Lift:3.453929904204978 5
=====================================
Rule:chewing gum->soda
Support:0.0004009891064626077 5
Confidence:0.3999999999999997
Lift:4.657743190661479
=====================================
Rule:citrus fruit->pork
Support:0.0004009891064626077 5
Confidence:0.3
Lift:3.4933073929961087
```

```
Confidence:0.2
Lift:3.3140642303433
=====================================
Rule:pip fruit->ice cream
Support:0.0003341575887188398
Confidence:0.5
Lift:4.545261239368165
=====================================
Rule:root vegetables->margarine
Support:0.0003341575887188398
Confidence:0.3333333333333333
Lift:4.791226384886326
=====================================
Rule:yogurt->onions
Support:0.0003341575887188398
Confidence:0.2777777777777778
Lift:3.2345438824038046
=====================================
Rule:waffles->sausage
Support:0.0003341575887188398
Confidence:0.4166666666666667
Lift:3.4124703521255246
=====================================
Rule:soft cheese->yogurt
Support:0.0003341575887188398
Confidence:0.2777777777777778
Lift:3.2345438824038046
=====================================
Rule:pork->sausage
Support:0.0004009891064626077 5
Confidence:0.2608695652173913
Lift:3.037658602605312
=====================================
Rule:whole milk->pastry
Support:0.0003341575887188398
Confidence:0.8333333333333334
Lift:5.276837353646496
=====================================
```

PRACTICAL NO 10

Demo of OpenAI/TensorFlow Tools

What is TensorFlow?

TensorFlow is an **open-source deep learning framework** developed by **Google Brain Team**.

It is widely used for **machine learning (ML)** and **deep learning (DL)** applications.

Core concept: *Tensor* (multi-dimensional data array) + *Flow* (computations through data flow graphs).

Key Features

Ease of Use – High-level Keras API for quick prototyping.

Flexibility – Works for deep learning, reinforcement learning, NLP, computer vision, etc.

Cross-Platform – Runs on CPUs, GPUs, TPUs, and even mobile devices.

Visualization – TensorBoard helps visualize model performance.

Scalability – Can handle large datasets and distributed training.

Applications

Computer Vision → image classification, object detection.

Natural Language Processing (NLP) → sentiment analysis, chatbots.

Healthcare → disease prediction, medical imaging.

Finance → fraud detection, stock prediction.

Robotics & IoT → real-time decision making.

What is OpenAI?

OpenAI is an AI research organization and company focused on building **safe, general-purpose artificial intelligence**.

Famous for **GPT (Generative Pretrained Transformer)** models → ChatGPT is based on GPT-4/GPT-5 family.

Provides APIs for NLP, vision, reasoning, and multi-modal tasks.

Key Features

Pretrained Models – No need for training from scratch.

Generative AI – Can generate human-like text, code, and images.

Zero-Shot & Few-Shot Learning – Works well even with little or no training data.

Multi-Modal – GPT-4o supports text, images, and even audio.

API Access – Easy integration into apps and products.

Applications

Chatbots & Virtual Assistants → customer support, education.

Content Generation → blogs, reports, code.

Language Translation → breaking communication barriers.

Creative Fields → story writing, music, art.

Data Analysis → summarization, classification, question answering.

Code :

```
# Import libraries
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt

# Step 1: Load Dataset (MNIST)
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# Normalize pixel values (0–255 → 0–1)
x_train = x_train / 255.0
x_test = x_test / 255.0

# Step 2: Build the Model
model = keras.Sequential([
  layers.Flatten(input_shape=(28,28)),      # Flatten 28x28 image → 784
  layers.Dense(128, activation="relu"),     # Hidden layer
  layers.Dropout(0.2),                      # Dropout to avoid overfitting
  layers.Dense(10, activation="softmax")    # Output layer (10 digits)
])

# Step 3: Compile the Model
model.compile(optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"])

# Step 4: Train the Model
history = model.fit(x_train, y_train, epochs=5, validation_split=0.1)

# Step 5: Evaluate Model
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"✅ Test Accuracy: {test_acc:.2f}")
```

```
# Step 6: Make Predictions
predictions = model.predict(x_test)

# Display a few test images with predictions
for i in range(5):
    plt.imshow(x_test[i], cmap="gray")
    plt.title(f"Predicted: {predictions[i].argmax()}, Actual: {y_test[i]}")
    plt.axis("off")
    plt.show()
```

Output :

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning:

Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8516 - loss: 0.5096 - val_accuracy: 0.9615 - val_loss: 0.1336
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 11s 5ms/step - accuracy: 0.9525 - loss: 0.1602 - val_accuracy: 0.9728 - val_loss: 0.0951
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9640 - loss: 0.1173 - val_accuracy: 0.9745 - val_loss: 0.0835
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9711 - loss: 0.0925 - val_accuracy: 0.9778 - val_loss: 0.0779
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 12s 6ms/step - accuracy: 0.9758 - loss: 0.0767 - val_accuracy: 0.9772 - val_loss: 0.0736
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9722 - loss: 0.0962
✅ Test Accuracy: 0.98
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
```

